My Notebook
tnseq_tutorial/tnseq analysis tutorial

PDF Version generated by

Matthew Davies

on

May 07, 2025 @08:04 PM CEST

Table of Contents

tnseq analysis tutorial 2

tnseq analysis tutorial

Matthew Davies - May 07, 2025, 8:04 PM GMT+2

Demultiplex the raw illumina output

- the tnseq output from illumina sequencer can't be demultiplexed with their pipeline so we need to do it ourselves with sabre https://github.com/najoshi/sabre
  - create barcode_file.txt of your barcodes + sample names, separated with a tab
    - can do this in command line text editor 'nano' ($ nano barcode_file.txt) and write it in manually, save with "ctrl o" and exit nano with "ctrl x"
  -
```
[mdavies@int4 tnseq]$ cat barcode_file.txt
TATAGCCT        CSF0_1.fastq
ATAGAGGC        CSF0_2.fastq
CCTATCCT        CSF0_3.fastq
GGCTCTGA        CSF4_1.fastq
AGGCGAAG        CSF4_2.fastq
TAATCTTA        CSF24_1.fastq
CAGGACGT        CSF24_2.fastq
GTACTGAC        CSF24_3.fastq
```
- demultiplex with sabre
  - install with conda or mamba into an environment then activate the environment. Sabre can be ran without much computing power so doesn't need to be sent to the slurm scheduler
  - `micromamba create -n sabre -c bioconda sabre`
  - `micromamba activate sabre`

```
sabre se -f Undetermined_S0_L001_R1_001.fastq.gz -b barcode_file.txt -u unknown_sabre_out.fastq
```

  - se for single end reads
  - -f is input file of the non-demultiplexed reads
  - -u output file for all the reads that the barcode is not detected in. The rest of the output will be in the files specified in the barcode file
- compare total number of non-demulitplexed reads with total number of reads with a barcode
  - For Sara and Jasper's run of the sequencer: Omdat 50% PhiX is toegevoegd verwacht je dat de ongeveer de helft van deze reads van jullie sample afkomstig is. Aligned is 61% wat wil zeggen dat het percentage PhiX in de data in werkelijkheid iets hoger is, namelijk 61%, maar dat betekend dat de opbrengst van jullie sample 39% van het totaal dus ongeveer 200 Mreads is.
  - so don't be concerned if the counts differ by a lot. For Marieke's project, there were 500M of initial raw reads and 300M with a barcode

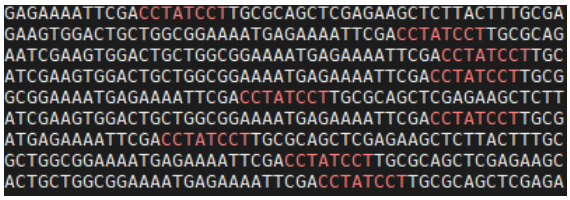
```
zcat Undetermined_S0_L001_R1_001.fastq.gz | wc -l
cat CSF*fastq | wc -l
```

- I've noticed that sabre seems to not correctly extract reads if the provided barcode starts at the second base. Check number of times this happens with:

```
grep "CCTATCCT" unknown_sabre_out.fastq | wc -l
grep "CCTATCCT" unknown_sabre_out.fastq
```

  - 'grep' searches a string within a file. If the file is gzipped, then you must zcat the file and pipe it (|) into grep for it to work correctly

normal (varies by barcode) suspicious





- determine if the number of reads with the barcode starting at the second base is too high. If so, we will trim the unknown_sabre_out.fastq of its first nucleotide and rerun the output through sabre a second time. I used seqtk (https://github.com/lh3/seqtk) for this and it also doesn't need to be sent to slurm
    - `micromamba deactivate`
    - `micromamba create -n seqtk -c bioconda seqtk`
    - `micromamba activate seqtk`

```
seqtk trimfq -b 1 unknown_sabre_out.fastq > unknown_sabre_out_trimmed.fastq
```

    - -b 1 means it will trim 1 bp from the left
- create an extra barcode file with different output names so that the first fastq files are not overwritten. E.g.:
    - TATAGCCT CSF0_1_extra.fastq
    - ATAGAGGC CSF0_2_extra.fastq
- rerun sabre but this time with unknown_sabre_out_trimmed as input

```
sabre se -f unknown_sabre_out_trimmed.fastq -b barcode_file_extra.txt -u
unknown_sabre_out_extra.fastq
```

- for each isolate, combine the outputs from the first and second runs of sabre and can remove the redundant files e.g:

```
cat CSF0_1.fastq CSF0_1_extra.fastq > CSF0_1_combined.fastq
rm CSF0_1.fastq CSF0_1_extra.fastq
```

    - easiest with a simple **for** loop:
    - `for var in *extra.fastq ; do cat ${var/_extra/} $var > ${var/_extra.fastq/_combined.fastq} ; done`

- unknown_sabre_out_extra.fastq should now contain no (or **far** fewer) suspicious reads. The output is quite large so to save space we will zip the files into .gz format with a for loop

```
for var in CSF*fastq ; do gzip $var ; done
```

Extract insertion site sequence

- the read should ideally begin with the 14bp of the gene of interest, but we need to still identify all of the reads with the correct adapter sequence at its 3' end. I used cutadapt for this (https://cutadapt.readthedocs.io/en/stable/guide.html)
    - `micromamba deactivate`
    - `micromamba create -n cutadapt -c bioconda cutadapt`
    - `micromamba activate cutadapt`

- Adapter sequence must start with 'TA' (the insertion site), so we need to ensure that only regions where the adapter has correctly inserted into are extracted. For this, I use TA + 6 nucleotides but if you want it to be more stringent you can make the screen for the adapter longer no problem

```
for var in CSF*fastq.gz ; do echo "cutadapt --cores 8 -a TAACAGGT --no-trim --untrimmed-output
${var/.fastq.gz/_no_adapter.fastq.gz} --error-rate=0 -o ${var/.fastq.gz/_untrimmed.fastq.gz} $var "
; done > commands_cutadapt1.txt
```

  - an error rate of 0 is to ensure that it's only sequences with a TA site
  - the flag --no-trim is included so that we don't remove the adapter sequence as we first only want to pull these reads out
  - I use 'echo' in the loop and save the output to a commands file, as this is more computationally heavy and it better to use in the slurm and just easier to parallelise this way
  - --cores 8 telling slurm to use 8 cpus as this is quicker. Be careful when parallelising as whatever number you give to the command "parralel -j # < commands_cutadapt1.txt" will be multiplied by 8

- check number of reads without a 3' adapter to see if it's suspiciously high or not

```
for var in *no_adapter.fastq.gz ; do zcat $var | wc - l ; done
```

  - Marieke's ranged from like 1 - 3 M reads

- rerun cutadapt on the reads that are positive for the 3' adapter so that it, and the subsequent nucleotides, are trimmed off

```
for var in *untrimmed.fastq.gz ; do echo "cutadapt --cores 8 -a ACAGGTTGGA --untrimmed-output
${var/_untrimmed*/_lowqual_reads.fastq.gz} --error-rate=0.1 -l -16 -m 14 -o
${var/_untri*/_transposon_site.fastq.gz} $var " ; done > commands_cutadapt2.txt
```

  - -l -16 to also trim the reads from the 5' end if the remaining read is longer than 16bp
    - negative 16 is so that it trims from the 5' end so that we have the 14bp of interest as well as preserving the TA tip
  - -m 14 is for minumum read length of 14

- check all of the fastq files now to see if the numbers and remaining %s are fairly consistent across samples and if you're happy with the output
  - note: each number for files containing **reads** are divided by 4 to calculate actual number of reads

```
for var in *fastq.gz ; do zcat $var | wc -l ; done
```

Align reads to reference genome

- now we have a fastq file of reads of ~14bp length that are **only** the gene sequence at the insertion site. This needs to be aligned against the reference strain in which these mutants were created to identify which genes were disrupted in each mutant
- we will use bowtie (https://bowtie-py.readthedocs.io/en/latest/quickstart.html) to align the reads to the reference genome but a bowtie database of the reference first needs to be made.

```
bowtie-build STAB902.fasta STAB902_bowtie
```

- then align to the reference bowtie database. Aligning is computationally heavy so will be sent to slurm

```
for var in *transposon_site.fastq.gz ; do echo "bowtie-align-s -S -v 0 -n 0 -p 8 -l 14 -m 1
STAB902/STAB902_bowtie $var ${var/.fastq.gz/_m1.sam} " ; done > commands_bowtie_m1.txt
```

  - -S writes output in sam format. These files are huge but the downstream analysis can't function with them gzipped, so gzip after you're done with them
  - -v 0 is telling it to report end-to-end hits with <= 0 mismatches
  - not sure if redundant but -n 0 is max number of mismatches = 0
    - -l 14 seed length of 14 for -n
  - -p number of threads. Be careful when parallelising when sending to
  - -m 1 suppresses alignments if >1 alignment exists. We run this so that only reads that align to unique regions of 14bp are detected. Otherwise it is impossible to determine which repeated region a read this small actually aligns to

- check the slurm log file to see how many reads are successfully aligning, how many have failed and how many are being suppressed due to the flag -m 1
    - for Sara and Marieke's, in the range of 15-25 million alignments are reported for each isolate
        - The % successful alignment is different between Sara and Marieke's but the final number of alignments is approx the same. Between 5-10% are suppressed due to -m
- **Ignore if not relevant** but it is possible to repeat the previous section but now removing all duplication in the reads and then no longer suppressing multiple alignment sites, instead aligning at random across the multiple hits. This way you're really looking at 1 read per potential insertion site but without problems of repeat regions
    - I didn't do this for the others' projects but the information may be informative for you if:
        - you are focussing on repeated regions
        - if a gene of interest (that isn't essential for survival even at wild type) may only have 1 or 2 TA insertion sites and is then not showing the expected number of aligned reads, possibly due to repeated 14bp elsewhere
    - Can use dedupe.sh script in BBMap (https://github.com/BioInfoTools/BBMap/tree/master) and align again with bowtie

```
dedupe.sh threads=4 in=transposon_site.fastq.gz out=transposon_site_dedupe.fastq.gz
bowtie-align-s -S -v 0 -n 0 -p 8 -l 14 -M 1 STAB902_analysis/STAB902_bowtie
transposon_site_dedupe.fastq.gz transposon_site_M1.sam
```

Quatradis statistics

- the alignment file (.sam) is now input into tradis/quatradis (https://github.com/quadram-institute-bioscience/QuaTradis). Clear installation instructions are on their github. Activate the micromamba environment
- start with creating an insertion site plot file from the .sam alignment file; it is a list of two columns indicating the nucleotide position and its hits
    - not computationally expensive so doesn't need to be sent to slurm

```
for var in *sam ; do tradis plot create -l $var -o ${var/_combined*/_tradis_out}
${var/_m1.sam/.fastq.gz} CJB111/CJB111.fasta ; done
```

- the insertion site plot file is used to calculate gene counts using an annotation embl file of the reference genome
    - I just used this website to convert genbank (full) to embl https://sequenceconversion.bugaco.com/converter/biology/sequences/genbank_to_embl.php

```
for var in *tradis_out/*plot.gz ; do tradis plot count CJB111/CJB111.embl $var ; done
```

- run statistics for each gene between the control group vs condition group
- there's a small bug where commas found in some gene annotations are treated as a delimiter during its analysis, so need to remove them first. Ideally, the annotation names in the embl file should be edited but it's easier to just edit the output *tradis_gene_insert_sites.csv for each instead.

```
for var in *tradis_out/*tradis_gene_insert_sites.csv ; do sed 's/,//g' $var -i ; done
```

    - 'sed' is used to quickly make edits to potentially every line of a file. It works by searching for a user-defined string per line and replacing it with another string. The s stands for substitute and the g for global, so it will perform the substitution any number of times the string of interest is found in a line, instead of only the first time. The string to search exists between the first //, so in this case is just a comma, and the substitution exists between the second //, so it is substituted with nothing and therefore is deleted. The flag -i just means to edit the file in place, remove this flag and it will print the output to screen which is helpful to check if it's working as expected; or you can instead save to a new file with "sed 's/,//g' $file > $newfile"

- number of control files and condition files need to be the same when calculating logFC and p+q values. Ideally at least 3 for each for sound statistics

```
tradis compare insertion_sites --controls
CSF0_1_combined_transposon_site.tradis_gene_insert_sites.csv
CSF0_2_combined_transposon_site.tradis_gene_insert_sites.csv
CSF0_3_combined_transposon_site.tradis_gene_insert_sites.csv --conditions
CSF24_1_combined_transposon_site.tradis_gene_insert_sites.csv
CSF24_2_combined_transposon_site.tradis_gene_insert_sites.csv
CSF24_3_combined_transposon_site.tradis_gene_insert_sites.csv --genome_length 2123900
CJB111/CJB111.embl --prefix logfc_0vs24 combined
```

    - change genome length to GAS
    - 'combined' is added so that it looks at both the forward and reverse strand genes

- the output combined.compare.csv file within logfc_0vs24 now contains logFC, logCPM and (corrected) p value results. Compare these results to the *tradis_gene_insert_sites.csv files from the repeats to see if the numbers are accurate and make sense
    - the logFC analysis has some internal filtering that it performs but I don't know the precise number. But it will only show results for genes with sufficient number of reads, which by-eye seems to be around at least an average of 100 reads

- Additionally, we can calculate gene essentiality for each condition and then compare between conditions. For Marieke's results, there seems to be genes that differ in essentiality between the conditions but these were not included in the logfc analysis

```
for var in *tradis_out/*insert_sites.csv ; do tradis compare essentiality $var ; done
```

- look in QC_and_changepoint_plots.pdf for ins_index cutoffs (**different per replicate**) that are used to calculate gene essentiality and ambiguity. This can then be overlay on your *tradis_gene_insert_sites.csv results
- There is the option 'tradis compare essentiality_analysis' to compare gene essentiality between control and condition but when I tried it, it uses the gene_name column in it's results. This is then a problem for any duplicate gene_names as the results file is then missing rows. Locus_tag is unique - and the output from 'tradis compare essentiality' uses this column - so I manually compare essentiality between control and condition instead.
    - the categories for 3 replicates are as below. The number in columns Control and Condition represent the number of times a gene's ins_index is below the essentiality cutoffs across the 3 repeats:

Gene	Essentiality	Control	Condition
gene1	always essential	3	3
gene2	always non-essential	0	0
gene3	probably always essential	3	2
		2	3
		2	2
gene4	probably always non-essential	1	0
		0	1
		1	1
gene5	essential in control	3	0
gene6	conditionally essential	0	3
gene7	probably essential in control	2	0
		2	1

		3	1
		0	2
gene8	inconsistent replicates	1	2
		1	3

- note: "probably essential in control" vs "inconsistent replicates" is up for debate and requires manually checking any gene of interest
    - there is more 'weight' put on essentiality in the control samples vs the condition samples,
        - given that they are not undergoing the same level of stress as the condition group. Inconsistent results between control replicates I therefore trust less, but it’s more a problem when the condition group is then essential
- always compare gene essentiality results to its read_count as sometimes it can switch from essential to non-essential just from a difference of a few reads
    - which in my opinion is not biologically relevant because a few reads is nothing at the larger scale, especially as all of the previous steps already filter out so many reads at each step
    - if, for example, a gene is non-essential at CSF0 and then essential at CSF24, but the read_counts at CSF0 are within the range of 100-1000, I personally wouldn't make a strong conclusion that this gene is then essential to survive in CSF **over time,** as it is already near-essential at CSF0.